Kamino Vault Rewards

Security Assessment

March 19th, 2026 — Prepared by OtterSec

Ajay Shankar Kunapareddy d1r3wolf@osec.io

Xiang Yin soreatu@osec.io

Tamta Topuria tamta@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-KVS-ADV-00 | Denial of Service via Overflow on Pending Rewards Calculation 6

OS-KVS-ADV-01 | Accounting Inconsistencies Due to Failure to Refresh Rewards 7

General Findings **9**

OS-KVS-SUG-00 | Unnecessary Mutable Payer Account 10

Appendices

Vulnerability Rating Scale **11**

Procedure **12**

© 2026 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Kamino Finance engaged OtterSec to assess the `native-vault-reward-streaming` feature. This assessment was conducted between February 24th and March 2nd, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 3 findings throughout this audit engagement.

In particular, we identified a vulnerability where a misconfigured per-second reward value may result in the pending rewards multiplication to overflow when refreshing rewards, resulting in core vault operations that call it to fail, and since configuration update paths also refresh rewards first, this prevents recovery via reconfiguration (OS-KVS-ADV-00). Furthermore, several reward-sensitive paths read or mutate vault accounting without invoking the reward-refreshing logic, so they may operate on stale available token amount and AUM, which skews investment invariants, allows retroactive accrual on newly topped-up rewards, and prices redemption against an outdated reward baseline (OS-KVS-ADV-01).

We also suggested removing an unnecessary constraint on a payer account to ensure it is not mutable.

© 2026 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/Kamino-Finance/kvault-private. The audit was performed against PR#78.

A brief description of the program is as follows:

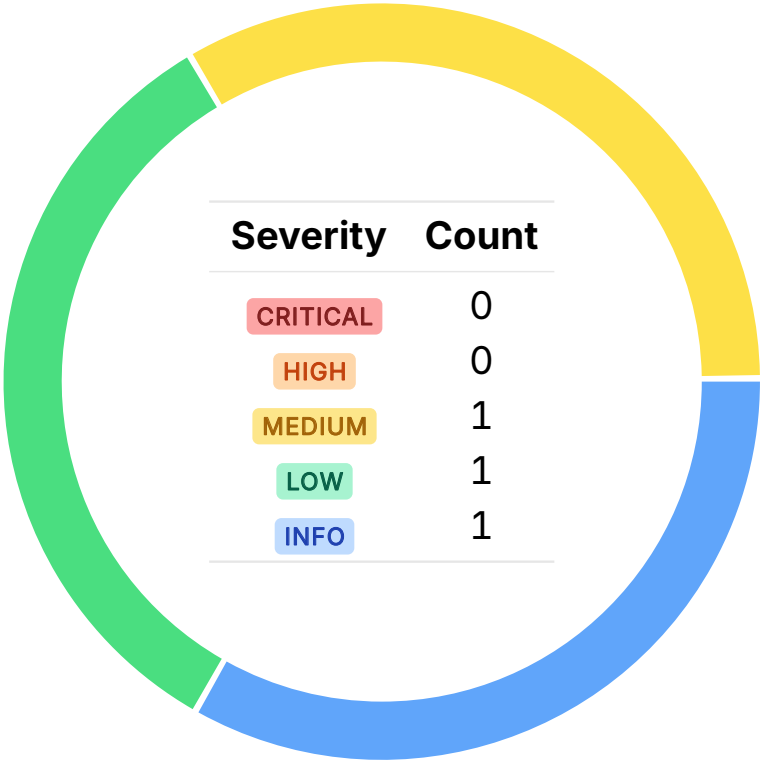
Name	Description
native-vault-reward-streaming	Introduces native same-token reward streaming to `Kvault`, allowing rewards to accrue within the vault and automatically increase share value over time instead of requiring separate claims. It adds vault reward state, refresh-on-interaction accounting, and management handlers for initializing, funding, and configuring the reward stream.

© 2026 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 3 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	1
LOW	1
INFO	1

© 2026 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-KVS-ADV-00	MEDIUM	RESOLVED	A misconfigured `reward_per_second` may result in the pending rewards multiplication to overflow when refreshing rewards, resulting in core vault operations that call it to fail, and since configuration update paths also refresh rewards first, this prevents recovery via reconfiguration.
OS-KVS-ADV-01	LOW	RESOLVED	Several reward-sensitive paths read or mutate vault accounting before calling `refresh_rewards` , so they may operate on stale available token amount and AUM. This skews investment invariants, allows retroactive accrual on newly topped-up rewards, and prices `redeem_in_kind` against an outdated reward baseline.

© 2026 Otter Audits LLC. All Rights Reserved.

Denial of Service via Overflow on Pending Rewards Calculation MEDIUM OS-KVS-ADV-00

Description

In `vault_operations::refresh_rewards`, when calculating `pending_rewards`, the multiplication of `reward_info.reward_per_second * seconds_passed` may overflow if `reward_per_second` is ever configured to an excessively large value (such as `u64::MAX`). Because `refresh_rewards` is invoked as a prerequisite in several core vault flows, including deposit and withdraw, any overflow-induced abort will propagate outward and block otherwise unrelated user actions, resulting in a denial-of-service across major user operations.

>_ *programs/kvault/src/operations/vault_operations.rs* RUST

```rust
pub fn refresh_rewards(vault: &mut VaultState, current_timestamp: u64) -> Result<u64> {
    let reward_info = &mut vault.reward_info;
    [...]
    let seconds_passed = current_timestamp.saturating_sub(reward_info.last_issuance_ts);
    if seconds_passed == 0 {
        return Ok(0);
    }

    let pending_rewards = reward_info.reward_per_second * seconds_passed;
    [...]
}
```

This issue is particularly severe because the admin recovery path (via `update_vault_config`) invokes `refresh_rewards` before applying the new configuration, resulting in the operation failing prior to state updates and thereby preventing recovery through reconfiguration.

Remediation

Utilize `saturating_mul` in the multiplication for calculating `pending_rewards` to avoid aborts due to overflow.

Patch

Resolved in 1dd1968.

© 2026 Otter Audits LLC. All Rights Reserved.

Accounting Inconsistencies Due to Failure to Refresh Rewards LOW OS-KVS-ADV-01

Description

There are several flows where reward-sensitive accounting is performed against a stale `vault.token_available` baseline. Since `refresh_rewards` moves vested rewards from `rewards_available` into immediately usable vault liquidity, any flow that reads holdings, AUM, or mutates reward state without refreshing may end up making decisions on outdated balances.

In the `invest` handler, `initial_holdings_total` is captured before `vault_operations::invest` runs, but `vault_operations::invest` itself calls `refresh_rewards` internally. Consequently, `initial_holdings_total` is based on a stale pre-refresh snapshot of `vault.token_available`. This is particularly problematic because the `invest` handler later utilizes those snapshots in `post_transfer_invest_checks`, weakening the precision of the accounting checks.

>_ *programs/kvault/src/handlers/handler_invest.rs* RUST

```rust
pub fn process<'info>(ctx: Context<'_, '_, '_, 'info, Invest<'info>>) -> Result<()> {
    [...]
    let initial_holdings_total =
        holdings(vault_state, reserves_iter.clone(), current_slot)?.total_sum;

    // Use vault_operations::invest directly which uses the holdings function internally
    let invest_effects = vault_operations::invest(
        vault_state,
        reserves_iter.clone(),
        &reserve,
        reserve_address,
        current_slot,
        current_timestamp,
        ctx.accounts
            .reserve_whitelist_entry
            .as_ref()
            .map(|acc| acc.as_ref()),
    )?;
    [...]
}
```

Additionally, in `vault_operations::topup_rewards`, the function adds to `rewards_available` without first settling rewards accrued up to `current_ts` by calling `refresh_rewards`. As a result, `last_issuance_ts` is not updated and still points to an older timestamp. Therefore, newly topped-up rewards are treated as if they had been present during the earlier elapsed time. This creates retroactive reward accrual when `refresh_rewards` is invoked subsequently.

© 2026 Otter Audits LLC. All Rights Reserved.

>_ *programs/kvault/src/operations/vault_operations.rs* RUST

```rust
pub fn topup_rewards(vault: &mut VaultState, amount: u64, current_ts: u64) -> Result<()> {
    require!(amount > 0, KaminoVaultError::RewardTopupAmountZero);
    vault.reward_info.rewards_available += amount;
    if vault.reward_info.last_issuance_ts == 0 && vault.reward_info.has_active_rewards() {
        vault.reward_info.last_issuance_ts = current_ts;
    }
    Ok(())
}
```

Also, in `vault_operations::redeem_in_kind`, rewards are not refreshed at all, so entitlement, penalty, and share-burn calculations may be based on stale pre-reward AUM.

Remediation

Invoke `refresh_rewards` before computing `initial_holdings_total` in `invest`, prior to mutating `rewards_available` in `topup_rewards`, and at the beginning of `redeem_in_kind`. More generally, any instruction that reads or mutates `token_available` or other reward-sensitive accounting state should invoke `refresh_rewards` first, ensuring that subsequent checks and accounting operate on an up-to-date baseline.

Patch

Resolved in dabe9a2.

© 2026 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-KVS-SUG-00	`payer` is only utilized to authorize the transfer and is not modified, so marking it `mut` is unnecessary in `TopupRewards`.

© 2026 Otter Audits LLC. All Rights Reserved.

Unnecessary Mutable Payer Account

OS-KVS-SUG-00

Description

In `TopupRewards`, `payer` is only utilized as the signing authority for the token transfer and is not itself modified by the instruction. Marking it `mut` therefore grants unnecessary write access.

>_ *programs/kvault/src/handlers/handler_topup_rewards.rs* RUST

```rust
#[derive(Accounts)]
pub struct TopupRewards<'info> {
    #[account(mut)]
    pub payer: Signer<'info>,
    [...]
}
```

Remediation

Remove the `mut` constraint on the `payer` account.

Patch

Resolved in 1dd1968.

© 2026 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.